10a. Overview and Goals

Martin Alfaro
PhD in Economics

The previous chapters established the foundations of performance optimization, focusing on strategies that apply broadly: type stability and memory allocation. This chapter transitions to **specialized optimization techniques**, by concentrating on **parallelization strategies**. The current chapter will be about parallelization within a single core, referred to as SIMD (Single Instruction, Multiple Data) or vectorization. The next chapter will extend the discussion to parallelization across multiple cores of a machine, known as multithreading.

While these two chapters will center on parallelization, there are some general lessons to carry forward. First, **advanced optimizations almost always involve trade-offs**. Once type stability is ensured and memory allocations are reduced, further speed gains almost always require a compromise. They may involve *reduced precision* (accepting less accurate results for faster computation), *diminished safety* (bypassing safeguards that prevent errors), or *limited generality* (writing code that is highly specific to one problem). This is precisely why such techniques aren't applied by default in Julia, which prioritizes correctness and safety above all.

The second lesson is **the use of macros to implement optimizations**. When the demand for performance justifies these trade-offs, macros provide an elegant way to implement them. By prefixing a code block with `@` and the macro name, developers can execute an operation using an optimized strategy. This makes advanced optimization highly accessible, letting users apply powerful techniques without requiring implementation knowledge.

The roadmap of the chapter is as follows:

Section 10b explains how to use macros. Their application goes beyond parallelization. Formally, macros take code blocks as input and transform them into new code blocks. In this way, macros are capable of implementing complex algorithms, but also of simplifying repetitive expressions and automating tasks.

In Section 10c, we start to explore vectorization by explaining conceptually what SIMD does. Basically, SIMD allows a single core to perform the same computation simultaneously across multiple data elements.

Importantly, not all operations are suitable for SIMD, whose implementation requires specific code structure. The subsequent sections establish the conditions necessary for efficient implementation:

- Section 10d: iterations must be independent. In other words, the computation in one iteration can't depend on another. The only exception is reductions, whose dependencies are handled automatically by SIMD.

- Section 10e: SIMD requires contiguous memory and unit strides (sequential access). Although non-contiguous layouts can sometimes be adapted, the cost of transforming the data may outweigh the benefits. A practical implication is that copies of slices, which guarantee contiguous blocks of memory, can outperform views, which avoid allocations but may involve non-contiguous access.
- Section 10f: SIMD requires branchless code. Importantly, branches aren't limited to explicit conditional statements, such as those involving `if`. Other operations, such as type instabilities and index checks, can introduce hidden branches through their internal computation.

Finally, Section 10g considers the package `LoopVectorization` and its macro `@turbo`. This implements more aggressive SIMD optimizations than Julia's default approach. However, such aggressive optimization can result in correctness issues if misapplied.